

Chapter 3: Elements of Dot net (.net) Languages

C# is part of .Net framework and is used for writing .Net applications.

3.1. The .Net Framework

The .Net framework is a revolutionary platform that helps us to write the following types of applications:

1. Windows applications
2. Web applications
3. Web services

The .Net framework applications are multi-platform applications. The framework has been designed in such a way that it can be used from any of the following languages: C#, C++, Visual Basic, Jscript, COBOL etc. All these languages can access the framework as well as communicate with each other.

The .Net framework consists of an enormous library of codes used by the client languages like C#. Following are some of the components of the .Net framework:

1. Common Language Runtime (CLR)
 - The .Net Framework Class Library
 - Common Language Specification (CLS)
 - Common Type System (CTS)
 - Metadata and Assemblies
 - Windows Forms
 - ASP.Net and ASP.Net AJAX
 - ADO.Net
2. Windows Workflow Foundation (WF)
 - Windows Presentation Foundation
 - Windows Communication Foundation (WCF)
 - LINQ

3.2. Integrated Development Environment (IDE) For C#

Microsoft provides the following development tools for C# programming:

3. Visual Studio 2010 (VS)
4. Visual C# 2010 Express (VCE)
5. Visual Web Developer

3.3. C# Program Structure

Before we study basic building blocks of the C# programming language, let us look at a bare minimum C# program structure.

C# Hello World Example

A C# program basically consists of the following parts:

- Namespace declaration
- A class
- Class methods

- Class attributes
- A Main method
- Statements & Expressions
- Comments

```
using System;
namespace HelloWorldApplication
{
    class HelloWorld
    {
        static void Main(string[] args)
        {
            /* my first program in C# */
            Console.WriteLine("Hello World");
            Console.ReadLine();
        }
    }
}
```

1. The first line of the program **using System;** - the using keyword is used to include the System namespace in the program. A program generally has multiple using statements.
2. The next line has the **namespace** declaration. A namespace is a collection of classes. The HelloWorldApplication namespace contains the class HelloWorld.
3. The next line has a class declaration, the class **HelloWorld**, contains the data and method definitions that the program uses. Classes generally would contain more than one method. Methods define the behavior of the class.
4. The next line defines the **Main** method, which is the entry point for all C# programs. The Main method states what the class will do when executed.
5. The next line */*...*/* will be ignored by the compiler and it has been put to add additional **comments** in the program.
6. The Main method specifies its behavior with the statement `Console.WriteLine("Hello World");`
7. `WriteLine` is a method of the `Console` class defined in the `System` namespace. This statement causes the message "Hello, World!" to be displayed on the screen.
8. The last line `Console.ReadLine();` makes the program wait for a key press and it prevents the screen from running and closing quickly when the program is launched from Visual Studio .NET.
9. Its worth to note the following points:
 - C# is case sensitive.
 - All statements and expression must end with a semicolon (;).
 - The program execution starts at the Main method.
 - Unlike Java, file name could be different from the class name.

Compilation

Type **csc helloworld.cs** and press enter to compile the code.

If there are no errors in our code the command prompt will take us to the next line and would generate helloworld.exe executable file.

Prepared by: Madan Kadariya
Nepal College of Information Technology

3.4. C# Basic Syntax

C# is an object oriented programming language. In Object Oriented Programming methodology a program consists of various objects that interact with each other by means of **actions**. The actions that an object may take are called **methods**. Objects of the same kind are said to have the same type or, more often, are said to be in the same class.

For example, let us consider a Rectangle object. It has attributes like length and width. Depending upon the design, it may need ways for accepting the values of these attributes, calculating area and display details.

```
using System;
namespace RectangleApplication
{
    class Rectangle
    {
        // member variables
        double length;
        double width;
        public void Acceptdetails()
        {
            length = 4.5;
            width = 3.5;
        }
        public double GetArea()
        {
            return length * width;
        }
        public void Display()
        {
            Console.WriteLine("Length: {0}", length);
            Console.WriteLine("Width: {0}", width);
            Console.WriteLine("Area: {0}", GetArea());
        }
    }
    class ExecuteRectangle
    {
        static void Main(string[] args)
        {
            Rectangle r = new Rectangle();
            r.Acceptdetails();
            r.Display();
            Console.ReadLine();
        }
    }
}
```

The **class** Keyword

The class keyword is used for declaring a class.

Prepared by: Madan Kadariya
Nepal College of Information Technology

Member Variables

Variables are attributes or data members of a class, used for storing data. In the preceding program, the Rectangle class has two member variables named length and width.

Member Functions

Functions are set of statements that perform a specific task. The member functions of a class are declared within the class. Our sample class Rectangle contains three member functions: AcceptDetails, GetArea and Display.

Instantiating a Class

In the preceding program, the class ExecuteRectangle is used as a class which contains the Main() method and instantiates the Rectangle class.

Identifiers

An identifier is a name used to identify a class, variable, function, or any other user-defined item. The basic rules for naming classes in C# are as follows:

1. A name must begin with a letter that could be followed by a sequence of letters, digits (0 - 9) or underscore. The first character in an identifier cannot be a digit.
2. It must not contain any embedded space or symbol like ? - + ! @ # % ^ & * () [] { } . ; : " ' / and \. However an underscore (_) can be used.
3. It should not be a C# keyword.

3.5. C# Keywords

Keywords are reserved words predefined to the C# compiler. These keywords cannot be used as identifiers, however, if we want to use these keywords as identifiers, we may prefix the keyword with the @ character. In C# some identifiers have special meaning in context of code, such as get and set, these are called contextual keywords.

The following table lists the reserved keywords in C#:

abstract	As	Base	bool	break	byte	case
catch	Char	checked	class	const	continue	decimal
default	delegate	do	double	else	enum	event
explicit	extern	false	finally	fixed	float	for
foreach	Goto	if	implicit	in	in	int
interface	internal	is	lock	long	namespae	new
null	object	operator	out	out	override	params
private	protected	public	readonly	ref	return	sbyte
sealed	short	sizeof	stackalloc	static	string	struct
switch	This	throw	true	try	typeof	uint
ulong	unchecked	unsafe	ushort	using	virtual	void
volatile			While			

3.6. C# Data types

In C#, variables are categorized into the following types:

1. Value types
2. Reference types

3. Pointer types

3.6.1 Value Types

Value type variables can be assigned a value directly. They are derived from the class **System.ValueType**.

The value types directly contain data. Some examples are **int**, **char**, **float**, which stores numbers, alphabets and floating point numbers respectively. When we declare an int type, the system allocates memory to store the value.

The following table lists the available value types in C#.

Type	Represents	Range	Default Value
bool	Boolean value	True or False	False
byte	8-bit unsigned integer	0 to 255	0
char	16-bit Unicode character	U +0000 to U +ffff	'\0'
decimal	128-bit precise decimal values with 28-29 significant digits	$(-7.9 \times 10^{28}$ to $7.9 \times 10^{28}) / 10^{0 \text{ to } 28}$	0.0M
double	64-bit double-precision floating point type	$(+/-)5.0 \times 10^{-324}$ to $(+/-)1.7 \times 10^{308}$	0.0D
float	32-bit single-precision floating point type	-3.4×10^{38} to $+3.4 \times 10^{38}$	0.0F
int	32-bit signed integer type	-2,147,483,648 to 2,147,483,647	0
long	64-bit signed integer type	-923,372,036,854,775,808 to 9,223,372,036,854,775,807	0L
sbyte	8-bit signed integer type	-128 to 127	0
short	16-bit signed integer type	-32,768 to 32,767	0
uint	32-bit unsigned integer type	0 to 4,294,967,295	0
ulong	64-bit unsigned integer type	0 to 18,446,744,073,709,551,615	0
ushort	16-bit unsigned integer type	0 to 65,535	0

To get the exact size of a type or a variable on a particular platform, we can use the **sizeof** method. The expression **sizeof(type)** yields the storage size of the object or type in bytes. Following is an example to get the size of int type on any machine:

```
namespace DataTypeApplication
```

```
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            Console.WriteLine("Size of int: {0}", sizeof(int));  
            Console.ReadLine();  
        }  
    }  
}
```

3.6.2 Reference Types

The reference types do not contain the actual data stored in a variable, but they contain a reference to the variables. In other words, they refer to a memory location. Using more than one variable, the

Prepared by: Madan Kadariya
Nepal College of Information Technology

reference types can refer to a memory location. If the data in the memory location is changed by one of the variables, the other variable automatically reflects this change in value. Examples of built in reference types are: **object, dynamic and string**.

Object Type

The Object Type is the ultimate base class for all data types in C# Common Type System (CTS). Object is an alias for **System.Object** class. So object types can be assigned values of any other types, value types, reference types, predefined or user-defined types. However, before assigning values, it needs type conversion.

When a value type is converted to object type, it is called boxing and on the other hand, when an object type is converted to a value type it is called unboxing.

```
object obj;  
obj = 100; // this is boxing
```

Dynamic Type

We can store any type of value in the dynamic data type variable. Type checking for these types of variables takes place at runtime.

Syntax for declaring a dynamic type is:

```
dynamic <variable_name> = value;
```

For example,

```
dynamic d = 20;
```

Dynamic types are similar to object types except that, type checking for object type variables takes place at compile time, whereas those for the dynamic type variables take place at run time.

String Type

The String Type allows us to assign any string values to a variable. The string type is an alias for the **System.String** class. It is derived from object type. The value for a string type can be assigned using string literals in two forms: quoted and @quoted.

For example,

```
String str = "First String";
```

A @quoted string literal looks like:

```
@"First String";
```

The user defined reference types are: class, interface, or delegate.

3.6.3 Pointer Types

Pointer type variables store the memory address of another type. Pointers in C# have the same capabilities as in C or C++.

Syntax for declaring a pointer type is:

```
type* identifier;
```

For example,

```
char* cptr;
```

```
int* iptr;
```

3.7 C# Type Conversion

Type conversion is basically type casting, or converting one type of data to another type. In C#, type casting has two forms:

1. **Implicit type conversion** - these conversions are performed by C# in a type-safe manner. Examples are conversions from smaller to larger integral types, and conversions from derived classes to base classes.
2. **Explicit type conversion** - users using the pre-defined functions do these conversions explicitly. Explicit conversions require a cast operator.

The following example shows an explicit type conversion:

```
namespace TypeConversionApplication
{
    class ExplicitConversion
    {
        static void Main(string[] args)
        {
            double d = 5673.74;
            int i;
            // cast double to int.
            i = (int)d;
            Console.WriteLine(i);
            Console.ReadLine();
        }
    }
}
```

Output:
5673

C# Type Conversion Methods

C# provides the following built-in type conversion methods:

1. **ToBoolean**: Converts a type to a Boolean value, where possible.
2. **ToByte** : Converts a type to a byte.
3. **ToChar**: Converts a type to a single Unicode character, where possible.
4. **ToDateTime**: Converts a type (integer or string type) to date-time structures.
5. **ToDecimal**: Converts a floating point or integer type to a decimal type.
6. **ToDouble**: Converts a type to a double type.
7. **ToInt16**: Converts a type to a 16-bit integer.
8. **ToInt32**: Converts a type to a 32-bit integer.
9. **ToInt64**: Converts a type to a 64-bit integer.
10. **ToSbyte**: Converts a type to a signed byte type.
11. **ToSingle**: Converts a type to a small floating-point number.
12. : Converts a type to a string.
13. : Converts a type to a specified type.
14. **ToUInt16**: Converts a type to an unsigned int type.
15. **ToUInt32**: Converts a type to an unsigned long type.
16. **ToUInt64**: Converts a type to an unsigned big integer.

The following example converts various value types to string type:

```
namespace TypeConversionApplication
{
```

```
class StringConversion
{
    static void Main(string[] args)
    {
        int i = 75;
        float f = 53.005f;
        double d = 2345.7652;
        bool b = true;
        Console.WriteLine(i.ToString());
        Console.WriteLine(f.ToString());
        Console.WriteLine(d.ToString());
        Console.WriteLine(b.ToString());
        Console.ReadLine();
    }
}
```

Output:

```
75
53.005
2345.7652
True
```

3.8. C# Variables

A variable is nothing but a name given to a storage area that our programs can manipulate. Each variable in C# has a specific type, which determines the size and layout of the variable's memory; the range of values that can be stored within that memory; and the set of operations that can be applied to the variable. We have already discussed various data types. The basic value types provided in C# can be categorized as:

Type	Example
Integral types	sbyte, byte, short, ushort, int, uint, long, ulong and char
Floating point types	float and double
Decimal types	decimal
Boolean types	true or false values, as assigned
Nullable types	Nullable data types

C# also allows defining other value types of variable like **enum** and reference types of variables like class.

Variable Declaration in C#

Syntax for variable declaration in C# is:

```
<data_type> <variable_list>;
```

Here, data_type must be a valid C# data type including char, int, float, double, or any user defined data type etc., and variable_list may consist of one or more identifier names separated by commas.

Some valid variable declarations along with their definition are shown here:

```
int i, j, k;
```



```
char c, ch;  
float f, salary;  
double d;
```

Variable Initialization in C#

Variables are initialized (assigned a value) with an equal sign followed by a constant expression. The general form of initialization is:

variable_name = value;

Variables can be initialized (assigned an initial value) in their declaration. The initializer consists of an equal sign followed by a constant expression as:

<data_type> <variable_name> = value;

Some examples are:

```
int d = 3, f = 5; /* initializing d and f. */  
byte z = 22; /* initializes z. */  
double pi = 3.14159; /* declares an approximation of pi. */  
char x = 'x'; /* the variable x has the value 'x'. */
```

Following example which makes use of various types of variables:

```
namespace VariableDeclaration  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            short a;  
            int b ;  
            double c;  
            /* actual initialization */  
            a = 10;  
            b = 20;  
            c = a + b;  
            Console.WriteLine("a = {0}, b = {1}, c = {2}", a,  
                b, c);  
            Console.ReadLine();  
        }  
    }  
}
```

Output:

a = 10, b = 20, c = 30

Accepting Values from User

The Console class in the System namespace provides a function ReadLine() for accepting input from the user and store it into a variable. int num;

```
num = Convert.ToInt32(Console.ReadLine());
```

The function *Convert.ToInt32()* converts the data entered by the user to int data type, because Console.ReadLine() accepts the data in string format.

Lvalues and Rvalues in C#:

There are two kinds of expressions in C#:

- 1. lvalue** : An expression that is an lvalue may appear as either the left-hand or right-hand side of an assignment.
- 2. rvalue** : An expression that is an rvalue may appear on the right- but not left-hand side of an assignment.

Variables are lvalues and so may appear on the left-hand side of an assignment. Numeric literals are rvalues and so may not be assigned and cannot appear on the left-hand side. Following is a valid statement:

```
int g = 20;
```

But following is not a valid statement and would generate compile-time error:

```
10 = 20;
```

3.9. C# Constants and Literals

The constants refer to fixed values that the program may not alter during its execution. These fixed values are also called literals. Constants can be of any of the basic data types like an integer constant, a floating constant, a character constant, or a string literal. There are also enumeration constants as well.

The constants are treated just like regular variables except that their values cannot be modified after their definition.

Integer Literals

An integer literal can be a decimal, octal, or hexadecimal constant. A prefix specifies the base or radix: 0x or 0X for hexadecimal, 0 for octal, and nothing for an integer literal can also have a suffix that is a combination of U and L, for unsigned and long, respectively. The suffix can be uppercase or lowercase and can be in any order.

Here are some examples of integer literals:

```
212 /* Legal */  
215u /* Legal */  
0xFeeL /* Legal */  
078 /* Illegal: 8 is not an octal digit */  
032UU /* Illegal: cannot repeat a suffix */
```

Following are other examples of various types of Integer literals:

```
85 /* decimal */  
0213 /* octal */  
0x4b /* hexadecimal */  
30 /* int */  
30u /* unsigned int */  
30l /* long */  
30ul /* unsigned long */
```

Floating-point Literals

A floating-point literal has an integer part, a decimal point, a fractional part, and an exponent part. We can represent floating point literals either in decimal form or exponential form.

Here are some examples of floating-point literals:

```
3.14159 /* Legal */  
314159E-5L /* Legal */
```

```
510E /* Illegal: incomplete exponent */  
210f /* Illegal: no decimal or exponent */  
.e55 /* Illegal: missing integer or fraction */
```

While representing using decimal form, we must include the decimal point, the exponent, or both and while representing using exponential form we must include the integer part, the fractional part, or both. The signed exponent is introduced by e or E.

Character Constants

Character literals are enclosed in single quotes e.g., 'x' and can be stored in a simple variable of char type. A character literal can be a plain character (e.g., 'x'), an escape sequence (e.g., '\t'), or a universal character (e.g., '\u02C0').

There are certain characters in C# when they are preceded by a back slash they will have special meaning and they are used to represent like newline (\n) or tab (\t). Here you have a list of some of such escape sequence codes:

Escape sequence	Meaning
\\	\ character
\'	' character
\"	" character
\?	? character
\a	Alert or bell
\b	Backspace
\f	Form feed
\n	Newline
\r	Carriage return
\t	Horizontal tab
\v	Vertical tab
\ooo	Octal number of one to three digits
\xhh . . .	Hexadecimal number of one or more digits

String Literals

String literals or constants are enclosed in double quotes "" or with @"". A string contains characters that are similar to character literals: plain characters, escape sequences, and universal characters.

We can break a long line into multiple lines using string literals and separating the parts using whitespaces.

Here are some examples of string literals. All the three forms are identical strings.

```
"hello, dear"  
"hello, \  
dear"  
"hello, " "d" "ear"  
@"hello dear"
```

3.10. Defining Constants

Constants are defined using the **const** keyword. Syntax for defining a constant is:

```
const <data_type> <constant_name> = value;
```

The following program demonstrates defining and using a constant in your program:

*Prepared by: Madan Kadariya
Nepal College of Information Technology*

```
using System;
namespace DeclaringConstants
{
    class Program
    {
        static void Main(string[] args)
        {
            const double pi = 3.14159; // constant declaration
            double r;
            Console.WriteLine("Enter Radius: ");
            r = Convert.ToDouble(Console.ReadLine());
            double areaCircle = pi * r * r;
            Console.WriteLine("Radius: {0}, Area: {1}", r,
                areaCircle);
            Console.ReadLine();
        }
    }
}
```

Output:

Enter Radius

3

Radius: 3, Area: 28.27431

3.11. C# Operators

An operator is a symbol that tells the compiler to perform specific mathematical or logical manipulations. C# is rich in built-in operators and provides the following type of operators:

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. Bitwise Operators
5. Assignment Operators
6. Misc Operators

Arithmetic Operators

Following table shows all the arithmetic operators supported by C#. Assume variable A holds 10 and variable B holds 20 then:

Operator	Description	Example
+	Adds two operands	A + B will give 30
-	Subtracts second operand from the first	A - B will give -10
*	Multiply both operands	A * B will give 200
/	Divide numerator by de-numerator	B / A will give 2
%	Modulus Operator and remainder of after an integer division	B % A will give 0
++	Increment operator increases integer value by one	A++ will give 11
--	Decrement operator decreases integer value by one	A-- will give 9

Relational Operators

Following table shows all the relational operators supported by C#. Assume variable A holds 10 and variable B holds 20 then:

Operator	Description	Example
==	Checks if the value of two operands is equal or not, if yes then condition becomes true.	(A == B) is not true.
!=	Checks if the value of two operands is equal or not, if values are not equal then condition becomes true.	(A != B) is true.
>	Checks if the value of left operand is greater than the value of right operand, if yes	(A > B) is not
	then condition becomes true.	true.
<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(A < B) is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(A >= B) is not true.
<=	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(A <= B) is true.

Logical Operators

Following table shows all the logical operators supported by C#. Assume variable A holds Boolean value true and variable B holds Boolean value false then:

Operator	Description	Example
&&	Called Logical AND operator. If both the operands are non zero then condition becomes true.	(A && B) is false.
	Called Logical OR Operator. If any of the two operands is non zero then condition becomes true.	(A B) is true.
!	Called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true then Logical NOT operator will make false.	!(A && B) is true.

Bitwise Operators

Bitwise operator works on bits and performs bit-by-bit operation. The truth tables for &, |, and ^ are as follows:

P	Q	p & q	p q	p ^ q
0	0	0	0	0
0	1	0	1	1
1	1	1	1	0
1	0	0	1	1

Assume if A = 60; and B = 13; Now in binary format they will be as follows:

A = 0011 1100

B = 0000 1101

A&B = 0000 1100

A|B = 0011 1101

A^B = 0011 0001

~A = 1100 0011

```
using System;
namespace OperatorsAppl
{
    class Program
    {
        static void Main(string[] args)
        {
            int a = 60; /* 60 = 0011 1100 */
            int b = 13; /* 13 = 0000 1101 */
            int c = 0;
            c = a & b; /* 12 = 0000 1100 */
            Console.WriteLine("Line 1 - Value of c is {0}", c
);
            c = a | b; /* 61 = 0011 1101 */
            Console.WriteLine("Line 2 - Value of c is {0}", c);
            c = a ^ b; /* 49 = 0011 0001 */
            Console.WriteLine("Line 3 - Value of c is {0}", c);
            c = ~a; /* -61 = 1100 0011 */
            Console.WriteLine("Line 4 - Value of c is {0}", c);
            c = a << 2; /* 240 = 1111 0000 */
            Console.WriteLine("Line 5 - Value of c is {0}", c);
            c = a >> 2; /* 15 = 0000 1111 */
            Console.WriteLine("Line 6 - Value of c is {0}", c);
            Console.ReadLine();
        }
    }
}
```

Output:

Line 1 - = Value of c = 21

Line 2 - += Value of c = 42
Line 3 - -= Value of c = 21
Line 4 - *= Value of c = 441
Line 5 - /= Value of c = 21
Line 6 - %= Value of c = 11
Line 7 - <<= Value of c = 44
Line 8 - >>= Value of c = 11
Line 9 - &= Value of c = 2
Line 10 - ^= Value of c = 0
Line 11 - |= Value of c = 2

Assignment Operators

Operator	Description	Example
=	Simple assignment operator, Assigns values from right side operands to left side operand	C = A + B will assign value of A + B into C
+=	Add AND assignment operator, It adds right operand to the left operand and assign the result to left operand	C += A is equivalent to C = C + A
-=	Subtract AND assignment operator, It subtracts right operand from the left operand and assign the result to left operand	C -= A is equivalent to C = C - A
*=	Multiply AND assignment operator, It multiplies right operand with the left operand and assign the result to left operand	C *= A is equivalent to C = C * A
/=	Divide AND assignment operator, It divides left operand with the right operand and assign the result to left operand	C /= A is equivalent to C = C / A
%=	Modulus AND assignment operator, It takes modulus using two operands and assign the result to left operand	C %= A is equivalent to C = C % A
<<=	Left shift AND assignment operator	C <<= 2 is same as C = C << 2
>>=	Right shift AND assignment operator	C >>= 2 is same as C = C >> 2
&=	Bitwise AND assignment operator	C &= 2 is same as C = C & 2
^=	bitwise exclusive OR and assignment operator	C ^= 2 is same as C = C ^ 2
=	bitwise inclusive OR and assignment operator	C = 2 is same as C = C 2

Misc Operators

There are few other important operators including sizeof, typeof and ?: supported by C#.

Operator	Description	Example
sizeof()	Returns the size of a data type.	sizeof(int), will return 4.
typeof()	Returns the type of a class.	typeof(StreamReader);
&	Returns the address of an variable.	&a; will give actual address of the variable.
*	Pointer to a variable.	*a; will pointer to a variable.
?:	Conditional Expression	If Condition is true ? Then value X : Otherwise value Y
is	Determines whether an object is of a certain type.	If(Ford is Car) // checks if Ford is an object of the Car class.
As	Cast without raising an exception if the cast fails.	Object obj = new StringReader("Hello"); StreamReader r = obj as StreamReader;

3.12. C# Decision Making

Decision making structures require that the programmer specify one or more conditions to be evaluated or tested by the program, along with a statement or statements to be executed if the condition is determined to be true, and optionally, other statements to be executed if the condition is determined to be false. C# provides following types of decision making statements.

3.12.1. If statement

An if statement consists of a boolean expression followed by one or more statements.

Syntax:

The syntax of an if statement in C# is:

```
if(boolean_expression)  
{  
/* statement(s) will execute if the boolean expression is true */  
}
```

If the boolean expression evaluates to true then the block of code inside the if statement will be executed. If boolean expression evaluates to false then the first set of code after the end of the if statement(after the closing curly brace) will be executed.

3.12.2. If...else statement

An if statement can be followed by an optional else statement, which executes when the boolean expression is false.

Syntax:

The syntax of an if...else statement in C# is:

```
if(boolean_expression)  
{  
/* statement(s) will execute if the boolean expression is true */  
}  
else  
{  
/* statement(s) will execute if the boolean expression is false */  
}
```

If the boolean expression evaluates to true then the if block of code will be executed otherwise else block of code will be executed.

3.12.3. The if...else if...else Statement

An if statement can be followed by an optional else if...else statement, which is very useful to test various conditions using single if...else if statement.

When using if , else if , else statements there are few points to keep in mind.

1. An if can have zero or one else's and it must come after any else if's.
2. An if can have zero to many else if's and they must come before the else.
3. Once an else if succeeds, none of the remaining else if's or else's will be tested.

Syntax:

The syntax of an if...else if...else statement in C# is:

```
if(boolean_expression 1)  
{  
/* Executes when the boolean expression 1 is true */
```

*Prepared by: Madan Kadariya
Nepal College of Information Technology*


```
}  
else if( boolean_expression 2)  
{  
/* Executes when the boolean expression 2 is true */  
}  
else if( boolean_expression 3)  
{  
/* Executes when the boolean expression 3 is true */  
}  
else  
{  
/* executes when the none of the above condition is true */  
}
```

3.12.4. nested if statements

It is always legal in C# to nest if-else statements, which means we can use one if or else if statement inside another if or else if statement(s).

Syntax:

The syntax for a nested if statement is as follows:

```
if( boolean_expression 1)  
{  
/* Executes when the boolean expression 1 is true */  
if(boolean_expression 2)  
{  
/* Executes when the boolean expression 2 is true */  
}  
}
```

3.12.5. Switch statement

A switch statement allows a variable to be tested for equality against a list of values. Each value is called a case, and the variable being switched on is checked for each switch case.

Syntax:

The syntax for a switch statement in C# is as follows:

```
switch(expression){  
    case constant-expression :  
        statement(s);  
        break; /* optional */  
    case constant-expression :  
        statement(s);  
        break; /* optional */  
    /* We can have any number of case statements */  
    default : /* Optional */  
        statement(s);  
}
```

The following rules apply to a switch statement:

1. The expression used in a switch statement must have an integral or enumerated type, or be of a class type in which the class has a single conversion function to an integral or enumerated type.

2. We can have any number of case statements within a switch. The value to be compared to and a colon follow each case.
3. The constant-expression for a case must be the same data type as the variable in the switch, and it must be a constant or a literal.
4. When the variable being switched on is equal to a case, the statements following that case will execute until a break statement is reached.
5. When a break statement is reached, the switch terminates, and the flow of control jumps to the next line following the switch statement.
6. Not every case needs to contain a break. If no break appears, the flow of control will fall through to subsequent cases until a break is reached.
7. A switch statement can have an optional default case, which must appear at the end of the switch. The default case can be used for performing a task when none of the cases is true. No break is needed in the default case.

3.12.6. The ? : Operator:

It has the following general form:

Exp1 ? Exp2 : Exp3;

Where Exp1, Exp2, and Exp3 are expressions. The value of a ? expression is determined like this: Exp1 is evaluated. If it is true, then Exp2 is evaluated and becomes the value of the entire ? expression. If Exp1 is false, then Exp3 is evaluated and its value becomes the value of the expression.

3.13. C# Loops

There may be a situation when we need to execute a block of code several number of times. In general statements are executed sequentially: The first statement in a function is executed first, followed by the second, and so on. Programming languages provide various control structures that allow for more complicated execution paths.

A loop statement allows us to execute a statement or group of statements multiple times and following is the general form of a loop statement in most of the programming languages:

3.13.1. While loop

A while loop statement in C# repeatedly executes a target statement as long as a given condition is true.

Syntax:

The syntax of a while loop in C# is:

```
while(condition)  
{  
    statement(s);  
}
```

Here statement(s) may be a single statement or a block of statements. The condition may be any expression, and true is any nonzero value. The loop iterates while the condition is true. When the condition becomes false, program control passes to the line immediately following the loop.

3.13.2. for loop

A for loop is a repetition control structure that allows us to efficiently write a loop that needs to execute a specific number of times.

Syntax:

The syntax of a for loop in C# is:

```
for ( init; condition; increment )  
{  
    statement(s);  
}
```

Here is the flow of control in a for loop:

1. The init step is executed first, and only once. This step allows us to declare and initialize any loop control variables. We are not required to put a statement here, as long as a semicolon appears.
2. Next the condition is evaluated. If it is true, the body of the loop is executed. If it is false, the body of the loop does not execute and flow of control jumps to the next statement just after the for loop.
3. After the body of the for loop executes, the flow of control jumps back up to the increment statement. This statement allows us to update any loop control variables. This statement can be left blank, as long as a semicolon appears after the condition.
4. The condition is now evaluated again. If it is true, the loop executes and the process repeats itself (body of loop, then increment step, and then again condition). After the condition becomes false, the for loop terminates.

3.13.3. do...while loop

Unlike for and while loops, which test the loop condition at the top of the loop, the do...while loop checks its condition at the bottom of the loop. A do...while loop is similar to a while loop, except that a do...while loop is guaranteed to execute at least one time.

Syntax:

The syntax of a do...while loop in C# is:

```
do  
{  
    statement(s);  
}while( condition );
```

If the condition is true, the flow of control jumps back up to do, and the statement(s) in the loop execute again. This process repeats until the given condition becomes false.

3.13.4. nested loops

C# allows to use one loop inside another loop.

Syntax:

The syntax for a nested for loop statement in C# is as follows:

```
for ( init; condition; increment )  
{  
    for ( init; condition; increment )  
    {  
        statement(s);  
    }  
    statement(s);  
}
```

Example: Program to find the prime numbers less than 100.

```
using System;
namespace Loops
{
    class Program
    {
        static void Main(string[] args)
        {
            /* local variable definition */
            int i, j;
            for (i = 2; i < 100; i++)
            {
                for (j = 2; j <= (i / j); j++)
                {
                    if ((i % j) == 0) break; // if factor found, not
                    prime
                }
                if (j > (i / j))
                {
                    Console.WriteLine("{0} is prime", i);
                }
                Console.ReadLine();
            }
        }
    }
}
```

3.14. Loop Control Statements:

Loop control statements change execution from its normal sequence. When execution leaves a scope, all automatic objects that were created in that scope are destroyed. C# provides the following control statements.

3.14.1. break statement

The break statement in C# has following two usage:

1. When the break statement is encountered inside a loop, the loop is immediately terminated and program control resumes at the next statement following the loop.
2. It can be used to terminate a case in the switch statement.

If we are using nested loops (ie. one loop inside another loop), the break statement will stop the execution of the innermost loop and start executing the next line of code after the block.

Syntax:

The syntax for a break statement in C# is as follows:

break;

3.14.2. continue statement

The continue statement in C# works somewhat like the break statement. Instead of forcing termination, however, continue forces the next iteration of the loop to take place, skipping any code in between. For the for loop, continue statement causes the conditional test and increment portions of the loop to execute. For the while and do...while loops, continue statement causes the program control passes to the conditional tests.

Syntax:

The syntax for a continue statement in C# is as follows:

continue;

3.15. C# Array

An array stores a fixed-size sequential collection of elements of the same type. An array is used to store a collection of data, but it is often more useful to think of an array as a collection of variables of the same type. All arrays consist of contiguous memory locations. The lowest address corresponds to the first element and the highest address to the last element.



3.15.1. Declaring Arrays

To declare an array in C#, you can use the following syntax:

```
datatype[] arrayName;
```

where,

1. *datatype* is used to specify the type of elements to be stored in the array.
2. *[]* specifies the rank of the array. The rank specifies the size of the array.
3. *arrayName* specifies the name of the array.

For example,

```
double[] balance;
```

3.15.2. Initializing and assigning values to an Array

Declaring an array does not initialize the array in the memory. When the array variable is initialized, we can assign values to the array. Array is a reference type, so we need to use the *new* keyword to create an instance of the array.

```
double[] balance = new double[10];
```

```
double[] balance = new double[10];
```

```
balance[0] = 4500.0;
```

1. We can assign values to the array at the time of declaration, like:

```
double[] balance = { 2340.0, 4523.69, 3421.0};
```
2. We can also create and initialize an array, like:

```
int [] marks = new int[5] { 99, 98, 92, 97, 95};
```
3. In the preceding case, we may also omit the size of the array, like:

```
int [] marks = new int[] { 99, 98, 92, 97, 95};
```
4. We can also copy an array variable into another target array variable. In that case, both the target and source would point to the same memory location:

```
int [] marks = new int[] { 99, 98, 92, 97, 95};  
int[] score = marks;
```

3.15.2. Accessing Array Elements

Indexing the array name accesses an element. Placing the index of the element within square brackets after the name of the array does this. For example:

```
double salary = balance[9];
```

3.15.3. Two-Dimensional Arrays:

The simplest form of the multidimensional array is the two-dimensional array. A two-dimensional array is, in essence, a list of one-dimensional arrays. A two dimensional array can be thought of as a table which will have x number of rows and y number of columns.

Initializing Two-Dimensional Arrays

Specifying bracketed values for each row may initialize multidimensional arrays. Following is an array with 3 rows and each row has 4 columns.

```
int [,] a = int [3,4] = {  
{0, 1, 2, 3} , /* initializers for row indexed by 0 */  
{4, 5, 6, 7} , /* initializers for row indexed by 1 */  
{8, 9, 10, 11} /* initializers for row indexed by 2 */  
};
```

Accessing Two-Dimensional Array Elements

Using the subscripts ie accesses an element in 2-dimensional array. row index and column index of the array. For example:

```
int val = a[2,3];
```

The above statement will take 4th element from the 3rd row of the array.

3.16. C# Classes

When we define a class, we define a blueprint for a data type. This doesn't actually define any data, but it does define what the class name means, that is, what an object of the class will consist of and what operations can be performed on such an object. Objects are instances of a class. The methods and variables that constitute a class are called members of the class.

3.16.1. Class Definition

A class definition starts with the keyword class followed by the class name; and the class body, enclosed by a pair of curly braces. Following is the general form of a class definition:

```
<access specifier> class class_name  
{  
    // member variables  
    <access specifier> <data type> variable1;  
    <access specifier> <data type> variable2;  
    ...  
    <access specifier> <data type> variableN;  
    // member methods  
    <access specifier> <return type> method1(parameter_list)  
    {  
        // method body  
    }  
    <access specifier> <return type> method2(parameter_list)  
    {  
        // method body  
    }  
}
```

```
...  
<access specifier> <return type> methodN(parameter_list)  
{  
    // method body  
}  
}
```

Please note that,

1. Access specifiers specify the access rules for the members as well as the class itself, if not mentioned then the default access specifier for a class type is **internal**. Default access for the members is **private**. Data type specifies the type of variable, and return type specifies the data type of the data, the method returns, if any.
2. To access the class members, we will use the dot (.) operator.
3. The dot operator links the name of an object with the name of a member.

3.16.2. Member Functions and Encapsulation

A member function of a class is a function that has its definition or its prototype within the class definition like any other variable. It operates on any object of the class of which it is a member, and has access to all the members of a class for that object.

Member variables are attributes of an object (from design perspective) and they are kept private to implement encapsulation. These variables can only be accessed using the public member functions.

The following example program can demonstrate this concept.

```
using System;  
namespace BoxApplication  
{  
    class Box  
    {  
        private double length; // Length of a box  
        private double breadth; // Breadth of a box  
        private double height; // Height of a box  
        public void setLength( double len )  
        {  
            length = len;  
        }  
        public void setBreadth( double bre )  
        {  
            breadth = bre;  
        }  
        public void setHeight( double hei )  
        {  
            height = hei;  
        }  
        public double getVolume()  
        {  
            return length * breadth * height;  
        }  
    }  
}
```

```
class Boxtester
{
    static void Main(string[] args)
    {
        Box Box1 = new Box(); // Declare Box1 of type Box
        Box Box2 = new Box(); // Declare Box2 of type Box
        double volume;
        // box 1 specification
        Box1.setLength(6.0);
        Box1.setBreadth(7.0);
        Box1.setHeight(5.0);
        // box 2 specification
        Box2.setLength(12.0);
        Box2.setBreadth(13.0);
        Box2.setHeight(10.0);
        // volume of box 1
        volume = Box1.getVolume();
        Console.WriteLine("Volume of Box1 : {0}" , volume);
        // volume of box 2
        volume = Box2.getVolume();
        Console.WriteLine("Volume of Box2 : {0}", volume);
        Console.ReadLine();
    }
}
```

Output:

Volume of Box1 : 210

Volume of Box2 : 1560

3.17. C# Collections

Collection classes are specialized classes for data storage and retrieval. These classes provide support for stacks, queues, lists, and hash tables. Most collection classes implement the same interfaces. Collection classes serve various purposes, such as allocating memory dynamically to elements and accessing a list of items on the basis of an index etc. These classes create collections of objects of the Object class, which is the base class for all data types in C#.

3.17.1. Various Collection Classes and Their Usage

The following are the various commonly used classes of the **System.Collection** namespace

1. **ArrayList** : It represents ordered collection of an object that can be indexed individually. It is basically an alternative to an array. However unlike array we can add and remove items from a list at a specified position using an index and the array resizes itself automatically. It also allows dynamic memory allocation, add, search and sort items in the list.
2. **Hashtable**: It uses a key to access the elements in the collection. A hash table is used when we need to access elements by using key, and we can identify a useful key value. Each item in the hash table has a key/value pair. The key is used to access the items in the collection.
3. **SortedList**: It uses a key as well as an index to access the items in a list. A sorted list is a combination of an array and a hash table. It contains a list of items that can be accessed using a key or an index. If we access items using an index, it is an ArrayList,

and if we access items using a key , it is a Hashtable. The collection of items is always sorted by the key value.

4. **Stack:** It represents a last-in, first out collection of object.
It is used when we need a last-in, first-out access of items. When we add an item in the list, it is called pushing the item and when you remove it, it is called popping the item.
5. **Queue:** It represents a first-in, first out collection of object.
It is used when we need a first-in, first-out access of items. When we add an item in the list, it is called enqueue and when we remove an item, it is called deque.
6. **Bitarray:** It represents an array of the binary representation using the values 1 and 0.
It is used when we need to store the bits but do not know the number of bits in advance. We can access items from the BitArray collection by using an integer index, which starts from zero.

Example of an ArrayList

```
using System;
using System.Collections;
namespace CollectionApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            ArrayList al = new ArrayList();
            Console.WriteLine("Adding some numbers:");
            al.Add(45);
            al.Add(78);
            al.Add(33);
            al.Add(56);
            al.Add(12);
            al.Add(23);
            al.Add(9);
            Console.WriteLine("Capacity: {0} ", al.Capacity);
            Console.WriteLine("Count: {0}", al.Count);
        }
    }
}
```

```
        Console.WriteLine("Content: ");
        foreach (int i in al)
        {
            Console.Write(i + " ");
        }
        Console.WriteLine();
        Console.WriteLine("Sorted Content: ");
        al.Sort();
        foreach (int i in al)
        {
            Console.Write(i + " ");
        }
        Console.WriteLine();
        Console.ReadKey();
    }
}
```

Output:

Adding some numbers:

Capacity: 8

Count: 7

Content: 45 78 33 56 12 23 9

Sorted Content: 9 12 23 33 45 56 78